Programming Assignment 2: Priority Scheduling

Student: B113040045 許育菖

https://glimmer-bass-dee.notion.site/Programming-Assignment-2-Priority-Scheduling-a1d76b2d81d44ce78e5c159976fc7e56?pvs=74

Basic lottery scheduling

修改的檔案有：**system.c、proc.h、proc.c**

- **system.c**

首先先新增標頭檔 `#include <stdlib.h>`

然後在函式 `system_init()` 開頭的地方設置亂數種子

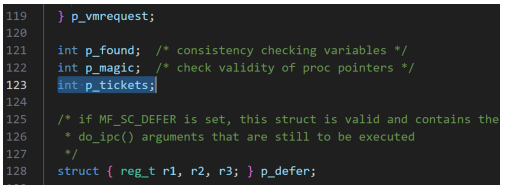
※注意此處不能使用 `srand(time(NULL))` ，不然 OS reboot 時會卡住

```
srandom( get_realtime() );
```

- **proc.h**

在struct proc中宣告一個變數用於紀錄每個 process 擁有的 tickets

```
int p_tickets; // new: process擁
```



- **proc.c**

新增標頭檔 `#include <stdlib.h>`

在函式 `proc_init()` 中，設定每個 process 初始擁有的tickets數量為 5

```
rp->p_tickets = 5;
```

修改函式 `pick_proc()` ，將每個 process 的票數相加起來，然後使用 `random()` 來隨機決定執行的 prcoess

```
static struct proc * pick_proc(void)
{

/* Decide who to run now.  A new process is selected an retur
 * When a billable process is selected, record it in 'bill_pt
 * clock task can tell who to bill for system time.
 *
 * This function always uses the run queues of the local cpu!
 */
    register struct proc *rp;   /* process to run */
    struct proc **rdy_head;
    struct proc *chosen = NULL;  // 最終選中的process

    int q;  /* iterate over queues */
    int total_tickets = 0;  // 總票數
  int lottery, current_ticket = 0;

 /* Check each of the scheduling queues for ready processes. 
    * queues is defined in proc.h, and priorities are set in
    * If there are no processes ready to run, return NULL.
    */
    rdy_head = get_cpulocal_var(run_q_head);
    for (q = 0; q < NR_SCHED_QUEUES; q++) {
        for (rp = rdy_head[q]; rp != NULL; rp = rp->p_nextrea
            if (proc_is_runnable(rp)) {
                total_tickets += rp->p_tickets;
            }
        }
    }

    // 如果没有就绪的process或彩票總數為0，return NULL
  if (total_tickets == 0)
      return NULL;

    lottery = random() % total_tickets;

    // 根據彩票结果選擇process
    for (q = 0; q < NR_SCHED_QUEUES; q++) {
```

```
        for (rp = rdy_head[q]; rp != NULL; rp = rp->p_nextrea
            if (proc_is_runnable(rp)) {
                current_ticket += rp->p_tickets;
                if (current_ticket > lottery) {
                    chosen = rp;
                    if (priv(rp)->s_flags & BILLABLE)
                        get_cpulocal_var(bill_ptr) = rp; // b
                    return chosen;
                }
            }
        }
    }

    return NULL;
}
```

Lottery scheduling with dynamic priorities

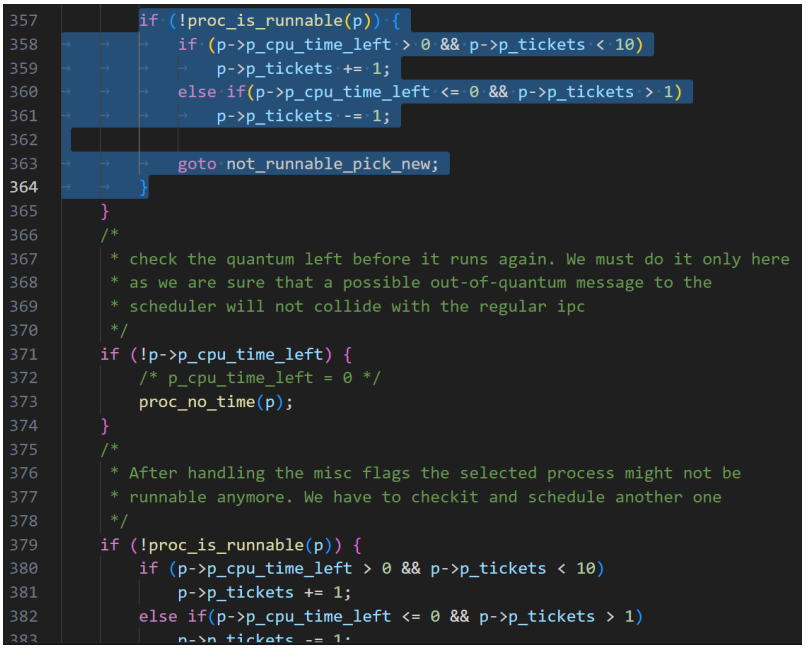
延續Basic lottery scheduling中所修改的檔案，並在**prco.c**中的函式`switch_to_user()`新增程式碼。

當 process 因為剩餘時間（p_cpu_time_left = 0）而中斷時，使當前 process 的 tickets 數量增加 1，最多增加到 10。

而當 process 執行完成，且還有剩餘時間（p_cpu_time_left > 0），使當前 process 的 tickets 數量減少 1，最少減少到 1。

```
if (!proc_is_runnable(p)) {
    if (p->p_cpu_time_left > 0 && p->p_tickets < 10)
        p->p_tickets += 1;
    else if(p->p_cpu_time_left <= 0 && p->p_tickets > 1)
        p->p_tickets -= 1;

    goto not_runnable_pick_new;
}
```



```
357         if (!proc_is_runnable(p)) {
358             if (p->p_cpu_time_left > 0 && p->p_tickets < 10)
359                 p->p_tickets += 1;
360             else if(p->p_cpu_time_left <= 0 && p->p_tickets > 1)
361                 p->p_tickets -= 1;
362 
363             goto not_runnable_pick_new;
364         }
365     }
366     /*
367      * check the quantum left before it runs again. We must do it only here
368      * as we are sure that a possible out-of-quantum message to the
369      * scheduler will not collide with the regular ipc
370      */
371     if (!p->p_cpu_time_left) {
372         /* p_cpu_time_left = 0 */
373         proc_no_time(p);
374     }
375     /*
376      * After handling the misc flags the selected process might not be
377      * runnable anymore. We have to checkit and schedule another one
378      */
379     if (!proc_is_runnable(p)) {
380         if (p->p_cpu_time_left > 0 && p->p_tickets < 10)
381             p->p_tickets += 1;
382         else if(p->p_cpu_time_left <= 0 && p->p_tickets > 1)
383             p->p_tickets -= 1;
```

line: 357 ~ 364

line: 379 ~ 386

詳細程式碼：

switch_to_user()

Profile your scheduler

在clock.c中宣告全域變數

```
static int interrupt_count = 0; // 發生interrupt的次數
static int priority[11] = {0}; // 1~10紀錄每個優先級擁有的程序數量
const int INTERRUPT_COUNT_MAX = 100; // interrupt_count = INT
```

並修改clock.c中的`timer_int_handler()`函式

在該函式一開頭的地方加上如下程式碼

```
int q;  /* iterate over queues */
struct proc **rdy_head;
register struct proc *rp;   /* process to run */

//  紀錄interrupt發生當下，每個優先級擁有的process數量
rdy_head = get_cpulocal_var(run_q_head);
for (q = 0; q < NR_SCHED_QUEUES; q++)
    if(rdy_head[q])
        for (rp = rdy_head[q]; rp != NULL; rp = rp->p_nextrea
            priority[ (rp->p_tickets) ]++;
```

```
//  紀錄發生了多少次interrupt
interrupt_count++;
if(interrupt_count % INTERRUPT_COUNT_MAX == 0)
{
    printf("INTERRUPT_COUNT: %d ~ %d\n", interrupt_count - 10
    int i;
    for(i = 0; i <= 10; i++)
    {
        printf("\ttickets %d: %d\n", i, priority[i]);
        priority[i] = 0; // reset
    }
}
```

這段程式碼是用來走訪每個process，並把它當前的tickets數紀錄起來

每當interrupt發生500次時，會把這段時間內所記錄到的tickets數output在螢幕上